

Infosys Springboard

Virtual Internship

Event Management System

A Comprehensive Full Stack Web Application

Tech Stack: Python, Django, SQLite, Bootstrap 5

Team A: Aadhil Nandan M, Akshay Kumar, Aryan Doshi,
Aryan Singh, Athulya R



Presentation Agenda

- Project Overview & Purpose
- Problem Statement & Objectives
- Technology Stack Selection
- System Architecture
- Database Design & ER Diagram
- Core Software Modules
- Application Workflow
- Implementation Details
- Application Showcase
- Software Testing Processes
- Development Challenges
- Future Scope & Conclusion

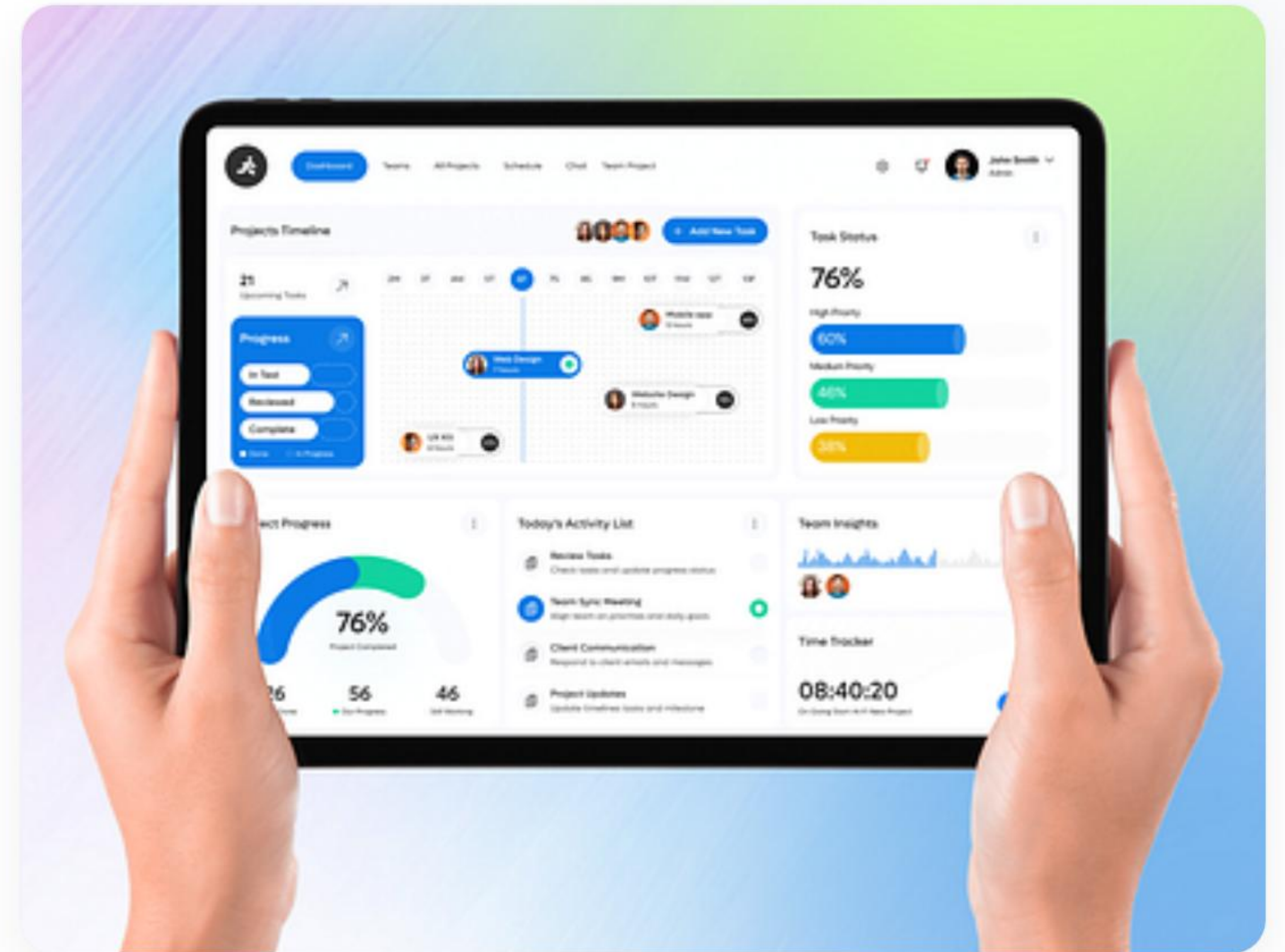
Project Overview

Modern Event Management

The Event Management System is a full-stack web application designed during the Infosys Springboard Virtual Internship to streamline the entire lifecycle of event planning and execution.

Key Deliverables

- **Purpose:** Centralize event planning, scheduling, and attendee tracking.
- **Target Users:** Event organizers, administrators, and participating members.
- **Benefits:** Reduces manual administrative workload and minimizes scheduling conflicts.



Problems & Objectives

Current Challenges

Traditional event management faces significant bottlenecks that hinder productivity:

- Time-consuming manual registrations
- Fragile paper-based record keeping
- Frequent scheduling conflicts
- Data inconsistency across departments
- Poor and delayed communication

Project Objectives

Our proposed system actively targets these pain points through digital transformation:

- Automate end-to-end event management
- Centralize and secure all event information
- Implement strict authentication protocols
- Improve administrative efficiency dramatically
- Simplify coordination for organizers

Technology Stack



Backend

The core logic is powered by **Python**, leveraging the robust **Django** framework to ensure rapid development, clean design, and high scalability.



Database

Data persistence is handled via **SQLite**, providing a lightweight, reliable, and zero-configuration relational database ideal for our prototype stage.



Frontend

The user interface is crafted using modern web standards including **HTML5** and **CSS3**, styled elegantly with the responsive **Bootstrap 5** library.

System Architecture

Django App

Handles URL routing, template rendering, and application middleware.

SQLite Database

Stores structured entities (Users, Events, Categories) safely via ORM.



Client Browser

Interacts with Frontend (HTML/Bootstrap), passing HTTP requests securely.



Views & Logic

Processes business logic, validates forms, and coordinates with Models.



Database Design

Entity Relationship Structure

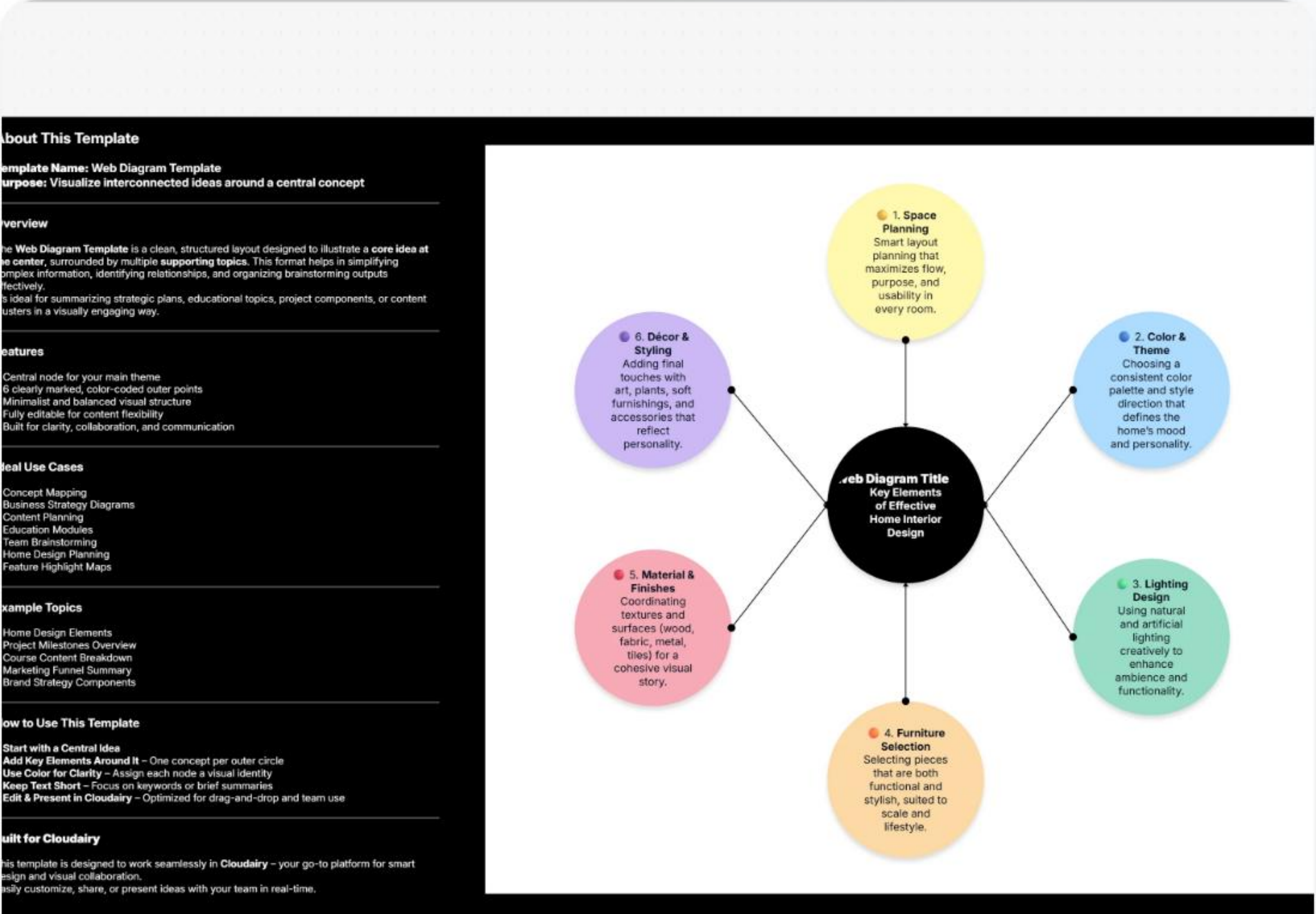
Our Django ORM maps directly to a relational SQLite structure, enforcing referential integrity and supporting complex queries.

 **User Entity:**
Handles secure authentication and permissions.

 **Category Entity:**
Organizes events into logical groupings.

 **Event Entity:**
Core data node containing schedules, venues, and constraints.

 **Member Entity:**
Tracks participant registrations and attendance.



Core Modules



Authentication

Secure login/logout and session management for users.



Dashboard

Centralized statistical overview of upcoming events.



Category Management

CRUD operations for various event types.



Event Management

Complete scheduling, updating, and event publication.



Member Management

Tracking attendee data, roles, and event assignments.



Reports Module

Generating insights, attendance metrics, and history.



Contact Module

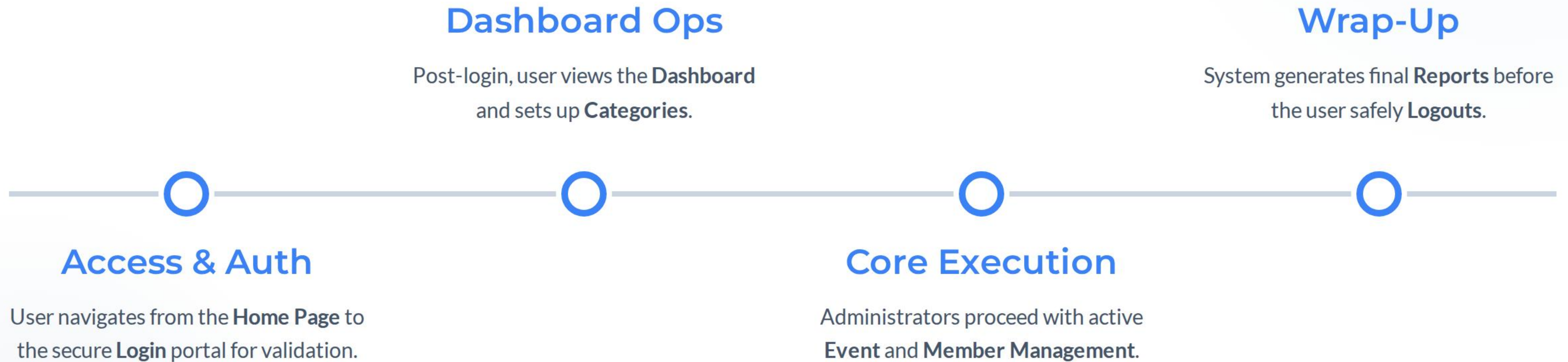
In-system communication and query management.



Profile Management

User preference updates and secure credential resets.

Application Workflow



Implementation Details

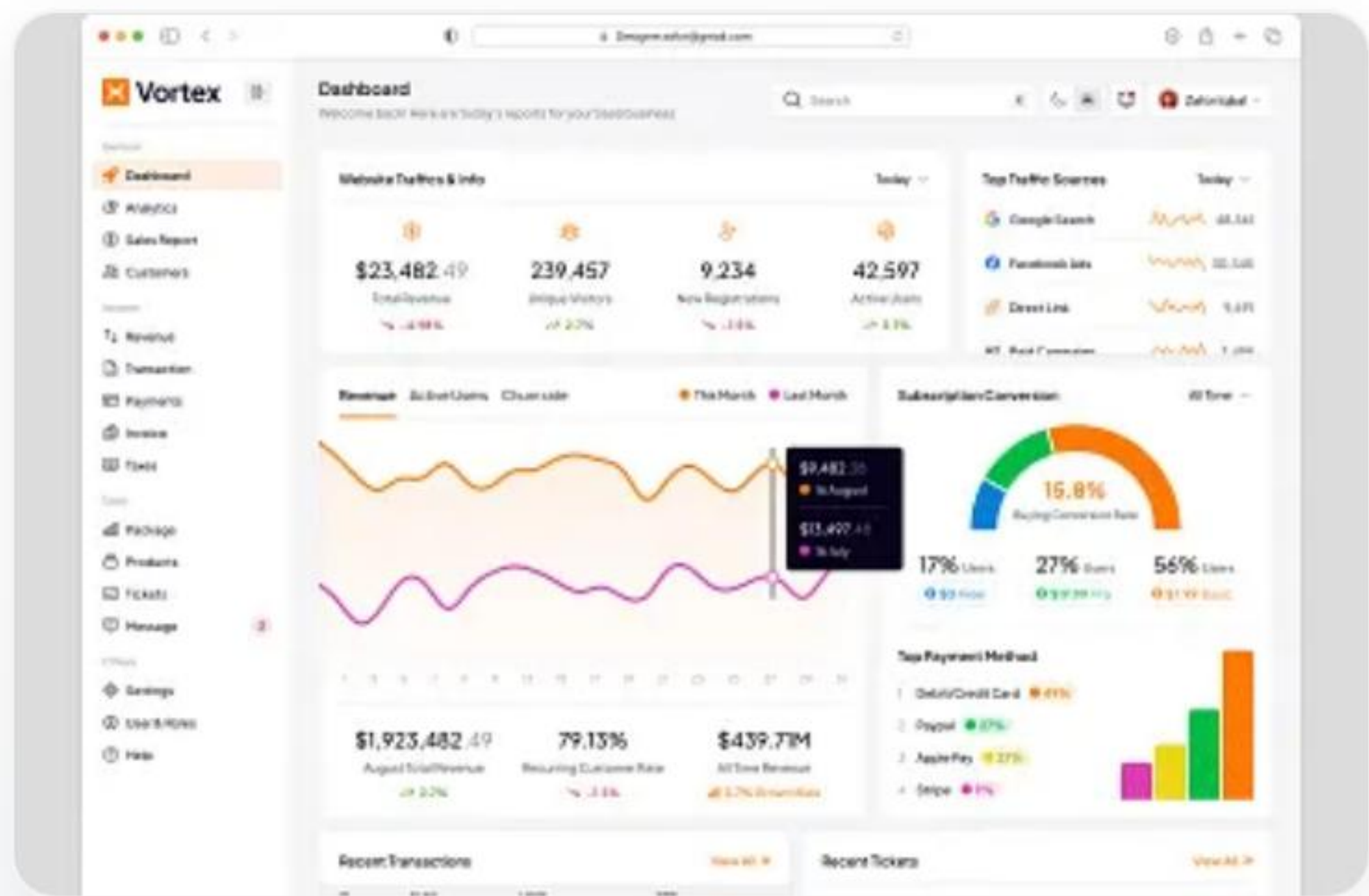
Architecture & Data

- **MVC/MTV Pattern:** Strictly decoupled Models, Templates, and Views for maintainability.
- **Django ORM:** Abstracted database operations preventing SQL injection.
- **SQLite Integration:** Seamlessly connected default database for local storage.
- **CRUD Operations:** Fully implemented across all primary system entities.

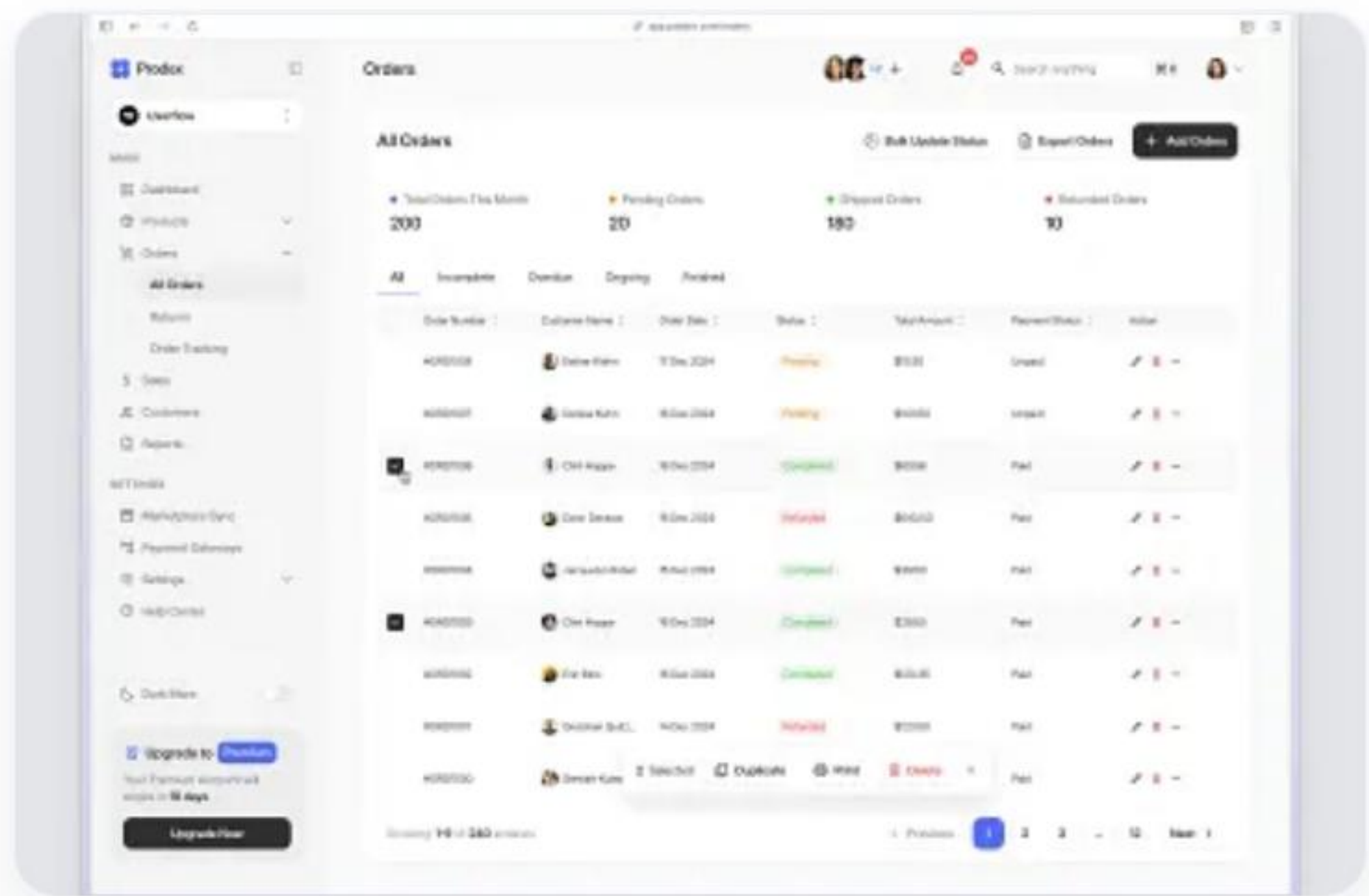
Security & Interface

- **Robust Authentication:** Leveraging Django's built-in, secure user auth system.
- **Form Validation:** Server-side data sanitization before database insertion.
- **Bootstrap Frontend:** Mobile-first responsive grids and components.
- **Static Media:** Properly configured serving of CSS, JS, and user uploads.

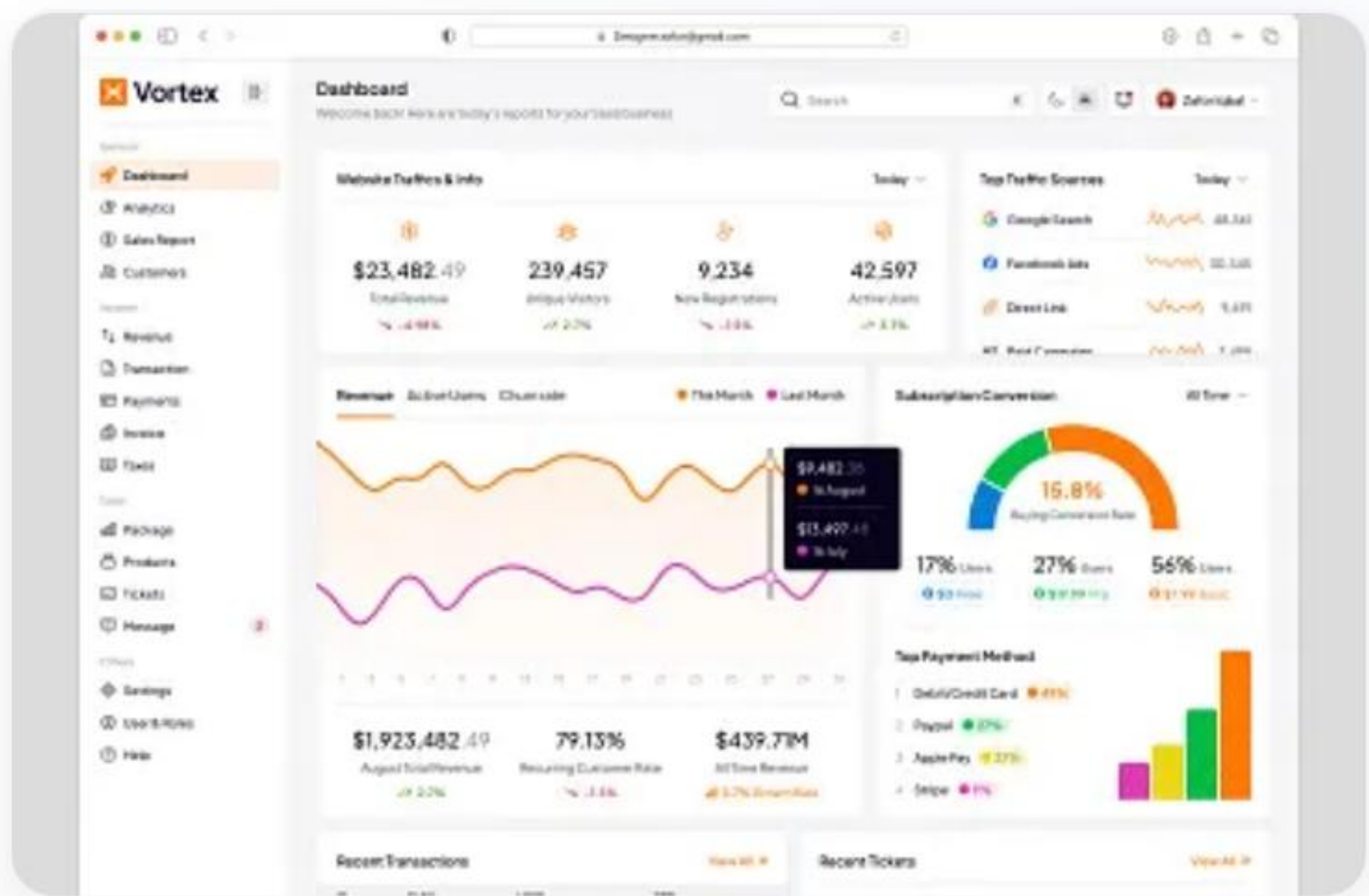
Application Showcase



Main Administrative Dashboard



Event & Member List View



Analytics & Reporting Module

Software Testing

TESTING PHASE	OBJECTIVE & METHODOLOGY	STATUS
Unit Testing	Verifying individual Django Models and isolated View logic functions.	✔ Passed
Functional Testing	Ensuring full CRUD operations perform correctly via the user interface.	✔ Passed
Integration Testing	Confirming modules (like Auth and Events) interact without data loss.	✔ Passed
Validation Testing	Testing form inputs against edge cases, invalid data, and empty fields.	✔ Passed

Challenges & Solutions



Challenge: Authentication Security

Solution: Implemented Django's native authentication middleware to ensure hashed passwords and secure session handling.



Challenge: Database Relationships

Solution: Utilized Django Models' ForeignKey to map Categories cleanly to Events without data redundancy.



Challenge: Form Validation Errors

Solution: Used Django ModelForms to automate validation, sending clean error messages back to the UI.



Challenge: User Interface Design

Solution: Adopted Bootstrap 5 to rapidly deploy a responsive, clean, and professional layout across all devices.

Future Scope



Advanced Features

Implementing **Role-Based Access Control (RBAC)** for granular permissions, alongside an **Online Payment Gateway** for paid ticketing.



Real-time Comms

Integrating **WebSockets** for real-time live chat between members, and enabling instant browser **Push Notifications** for updates.



External Integration

Adding direct **Calendar Synchronization** allowing users to automatically export event schedules to Google Calendar or Outlook.

Project **Summary**

Thank You for Your Time & Attention!

We are now open for Questions & Answers.

Developed by Team A | Infosys Springboard

Aadhil Nandan M • Akshay Kumar • Aryan Doshi

Aryan Singh • Athulya R

Image Sources



<https://cdn.outrank.so/aca2e26f-b928-4141-ad69-bfd0d915eeb3/7f947e58-c06e-498c-a376-4193b7a41609/software-development-team-structure-team-meeting.jpg>

Source: group107.com



<https://cdn.dribbble.com/userupload/48572925/file/2d4b4993db0fdb1750aad6e0cb2b96f4.png?resize=420x&vertical=center>

Source: dribbble.com



[https://cdn.cloudairy.com/template/1762513690679h15%20\(1\).webp](https://cdn.cloudairy.com/template/1762513690679h15%20(1).webp)

Source: cloudairy.com



<https://cdn.dribbble.com/userupload/44730972/file/0849bd58c815915ac1798dd5ef181bc2.jpg?crop=0x0-3201x2401&format=webp&resize=400x300&vertical=center>

Source: dribbble.com



Thumbnail

for

dribbble.com

<https://cdn.dribbble.com/userupload/24363964/file/original-d644e753f2fa9946fae24afdf7c9c0f6.jpg?format=webp&resize=400x300&vertical=center>

Source: dribbble.com